SESSION 6: DATA STRUCTURES - 2

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what sets are.
- Understand common methods and operations associated with sets.
- Understand comparison between lists, tuples and sets
- Understand what dictionaries are
- Understand common methods and operations associated with dictionaries.
- Understand comparison between lists, tuples, sets and dictionaries.

KEY POINTS AND EXAMPLES:

Understand what sets are

A set in Python is unordered, changeable, unindexed, and do not allow duplicate elements. It can store a collection of items of different data types in single variable.

- **Unordered:** Sets do not maintain the order of elements as they are added. Unlike sequences like lists or tuples, where elements are indexed and ordered, sets are unordered collections.

- **Changeable:** Sets in Python are mutable, meaning you can add or remove elements from them. However, the **elements themselves are immutable**. Once added to a set, the individual elements cannot be changed.The term "immutable" refers to the fact that you cannot change the elements of a set through indexing like you can with lists or tuples.
- **Unindexed:** Sets do not support indexing. You cannot access elements in a set using indices like you can with lists or tuples. This is because sets are unordered collections, and there is no guaranteed order in which elements are stored.

- **Duplicates Not Allowed:** Each element in a set must be unique. When you try to add a duplicate element to a set, it will simply ignore the duplicate and keep only one instance of it.

Creating Sets: Sets in Python can be created using curly braces {}, with elements separated by commas.

```
set1 = set()  # Empty set
set2 = {1, 2, 3}  # Set of integers
set3 = {1, "One", 3.4} # Set with mixed data types
```

```
# Creating a set with duplicate values
set_with_duplicates = {1, 2, 2, 3, 3, 3}
print(set_with_duplicates)
```

The output will be:

```
{1, 2, 3}
```

In this example, even though we specify duplicate values (2 and 3), the resulting set set_with_duplicates only contains unique elements. Duplicate values are automatically removed when creating a set.

```
thisset = {"apple", "banana", "cherry", False, True, 0}
print(thisset)
```

The output will be:

```
{'banana', 'cherry', 'apple', False, True}
```

In Python, the boolean value False and the integer value 0 are considered to be the same value when it comes to sets. For boolean values, False and True have hash values of 0 and 1, respectively. Since 0 is the hash value for both False and the integer 0, they are considered the same value in the context of a set.

Sets in Python, like lists, do not require a comma after a single element.

```
# Creating a set with a single element
single_element_set = {42}
print(single_element_set)
```

The output will be:

```
{42}
```

The set() Constructor: It is also possible to use the set() constructor to make a set. Using constructor we can convert one data structure to another.

```
my_list = [1, 2, 3, 4, 5]
my_set = set(my_list)
print(my_set)  # Output: {1, 2, 3, 4, 5}
```

```
my_tuple = (1, 2, 3, 4, 5)
my_set = set(my_tuple)
print(my_set)  # Output: {1, 2, 3, 4, 5}
```

```
my_string = "hello"
my_set = set(my_string)
print(my_set)  # Output: {'e', 'h', 'l', 'o'}
```

Note: Sets do not retain the original order of elements, and duplicate elements are automatically removed.

We can also use the type() function with sets:

```
print(type(my_set))
```

The output will be:

```
<class 'set'>
```

Sets in Python cannot have nested sets directly. However, we can create a set of tuples, and those tuples can contain sets.

```
# Creating a set of tuples, where each tuple contains a set
nested_set = {('a', 'b', 'c'), ('d', 'e', 'f', 'g'), ('h', 'i')}
print(nested_set)
```

The output will be:

```
{('d', 'e', 'f', 'g'), ('h', 'i'), ('a', 'b', 'c')}
```

In this example, nested_set contains three tuples, and each tuple contains elements that are

themselves sets. While this isn't exactly a nested set structure, it allows you to work with sets within tuples.

We can create lists inside sets in Python. However, it's important to note that lists are mutable, and mutable objects cannot be elements of sets. This means that while we can technically create a set that contains lists, it's not a recommended practice because sets require their elements to be immutable.

Creating a set with a list as an element

```
my_set = {['a', 'b', 'c']}
print(my_set)
```

The output will be:

```
TypeError: unhashable type: 'list'
```

Understand common methods and operations associated with sets

Access Set Items: We cannot access items in a set by referring to an index but we can use membership operators such as in and not in to check if an item exists in a set. These operators allow you to efficiently check for the presence or absence of elements in a set without needing to access them by index.

```
thisset = {"apple", "banana", "cherry"}
print("banana" in thisset)
```

The output will be:

```
True
```

Length of a Set: To determine how many items a set has, use the len() function.

```
set1 = {"abc", 34, True, 40, "male"}
print(len(set1))
```

The output will be:

```
5
```

Finding minimum or maximum element in a set: To find the minimum or maximum element in a set in Python, you can use the min() and max() functions, respectively. These functions work similarly to how they work with lists and tuples.

```
my_set = {3, 1, 4, 1, 5, 9, 2, 6, False, True}
# Find the minimum element in the set
min_element = min(my_set)
print(min_element)
# Find the maximum element in the set
max_element = max(my_set)
print(max_element)
```

The output will be:

```
False
9
```

The same principle applies to sets , boolean values (True and False) would be compared based on their numeric equivalents (False as 0 and True as 1) when present along with numeric types in sets.

In this example, min(my_set) returns the minimum element in the set my_set, while max(my_set) returns the maximum element. If the set contains non-comparable elements (e.g., a mixture of integers and strings), attempting to find the minimum or maximum element will result in a TypeError.

sum of all elements in a set: Just like lists and tuples, the sum() function in Python can also be used with setss to calculate the sum of all numeric elements. However, the elements within the sets must be numeric for this to work.

```
my_set = {6, 2, 3, 4, 5, True}
total = sum(my_set)
print(total)
```

The output will be:

```
21
```

Add Set Items: Adding items to a set in Python can be done using the add() and update() methods.

1. **add() Method:** The add() method adds a single element to the set. If the element is already present in the set, the set remains unchanged.

Syntax:

```
set.add(element)
```

```
my_set = {1, 2, 3}
my_set.add(4)
print(my_set)
```

The output will be:

```
{1, 2, 3, 4}
```

The add() method in the example adds the element 4 to the set, but it's not necessarily guaranteed to be added at the end of the set. The position of the added element within the set is not defined, as sets in Python are unordered collections. Therefore, the element could be added anywhere within the set.

2. **update() Method:** The update() method adds multiple elements to the set. It takes an iterable (such as a list, tuple, or another set) as an argument.If any of the elements being added are already present in the set, they are ignored.

Syntax:

```
set.update(iterable)
```

```
my_set = {1, 2, 3}
my_set.update([4, 5, 6])
print(my_set)
```

The output will be:

```
{1, 2, 3, 4, 5, 6}
```

lists are mutable and sets require their elements to be immutable. but If the iterable contains mutable objects like lists, those lists themselves are not added to the set. Instead, the individual elements of the list are added to the set.

Remove Set Items: Removing items from a set in Python can be done using various methods.

1. **remove():** This method removes the specified element from the set. If the element is not present, it raises a KeyError.

```
my_set = {1, 2, 3, 4, 5}
my_set.remove(3)
print(my_set)
```

The output will be:

```
{1, 2, 4, 5}
```

```
my_set = {1, 2, 3, 4, 5}
my_set.remove(6)
print(my_set)
```

The output will be:

```
KeyError: 6
```

2. **discard():** This method removes the specified element from the set if it is present. If the element is not present, it does nothing.

```
my_set = {1, 2, 3, 4, 5, "banana"}
my_set.discard(3)
print(my_set)
```

The output will be:

```
{1, 2, 4, 5, 'banana'}
```

```
my_set = {1, 2, 3, 4, 5, "banana"}
my_set.discard(6)
print(my_set)
```

The output will be:

```
{1, 2, 3, 4, 5, 'banana'}
```

3. **pop():** This method removes and returns an arbitrary element from the set meaning any element from the set can be removed, but there is no guarantee about which specific element will be removed. If the set is empty, it raises a KeyError. It's important not to rely on any specific ordering when using pop() with sets.

```
thisset = {"apple", "banana", "cherry"}
x = thisset.pop()
print(x) # removed item
print(thisset) # the set after removal
```

The output will be:

```
apple
{'banana', 'cherry'}
```

4. **clear():** This method removes all elements from the set, resulting in an empty set.

```
my_set = {1, 2, 3, 4, 5}
my_set.clear()
print(my_set)
```

The output will be

```
set()
```

5. **del keyword:** The del keyword will delete the set completely

```
thisset = {"apple", "banana", "cherry"}
del thisset
print(thisset)
```

This will raise an error because the set no longer exists

The output will be:

```
NameError: name 'thisset' is not defined
```

Join Sets:The "Join Sets" operation in Python involves combining elements from two or more sets into a single set. This can be achieved using various set methods.

1. **Union (|):** The union of two sets A and B contains all elements that are present in either set A, set B, or both. It's represented by the | operator or the union() method.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
union_set = set1 | set2  # or set1.union(set2)
print(union_set)  # Output: {1, 2, 3, 4, 5}
```

   The union() and update() methods joins all items from both sets.
   Note: the `+` operator is not supported for set concatenation. Instead union is used in sets for concatenation.

2. **Intersection (&):** The intersection of two sets A and B contains only the elements that are common to both sets. It's represented by the & operator or the intersection() method.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
intersection_set = set1 & set2  # or set1.intersection(set2)
print(intersection_set)  # Output: {3}
```

3. **Difference (-):** The difference between two sets A and B contains the elements that are present in set A but not in set B. It's represented by the - operator or the difference() method.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
difference_set = set1 - set2  # or set1.difference(set2)
print(difference_set) # Output: {1,2}
```

4. **Symmetric Difference (^):** The symmetric difference between two sets A and B contains the elements that are present in either set A or set B, but not in both. It's represented by the ^ operator or the symmetric_difference() method.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
symmetric_difference_set = set1 ^ set2  # or
```

```
set1.symmetric_difference(set2)
print(symmetric_difference_set)  # Output: {1, 2, 4, 5}
```

Copying Sets: Copying sets in Python involves creating a new set with the same elements as an existing set. There are a few different ways to copy sets:

1. **Using the copy() Method:** The copy() method creates a shallow copy of the set. Changes made to the original set will not affect the copied set, and vice versa.

```
original_set = {1, 2, 3}
copied_set = original_set.copy()
```

2. **Using the set() Constructor:** You can use the set() constructor to create a new set from an existing set. This also creates a shallow copy.

```
original_set = {1, 2, 3}
copied_set = set(original_set)
```

Common Set Methods: There are various methods available for working with sets

1. **difference_update():** Removes the items in this set that are also included in another, specified set. As a shortcut, we can use the -= operator.

```
a = {"apple", "banana", "cherry"}
b = {"google", "microsoft", "apple"}
c = {"cherry", "micra", "bluebird"}
a.difference_update(b, c)
print(a) # Output: {"banana"}
```

2. **intersection_update():** Removes the items in this set that are not present in other specified sets. As a shortcut, we can use the &= operator

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.intersection_update(y)
print(x) # Output: {"apple"}
```

3. **isdisjoint():** The isdisjoint() method in Python is used to check whether two sets are disjoint or not. Two sets are said to be disjoint if they have no common elements. The method returns True if the sets have no common elements (i.e., they are disjoint),

otherwise it returns False.

```
set1 = {1, 2, 3}
set2 = {4, 5, 6}
# Check if set1 and set2 are disjoint
result = set1.isdisjoint(set2)
print(result)  # Output: True
```

4. **issubset():** The issubset() method in Python is used to check whether a set is a subset of another set. A set A is considered a subset of another set B if all elements of A are also present in B.As a shortcut, we can use the <= operator.

```
set1 = {1, 2, 3}
set2 = {1, 2, 3, 4, 5}
# Check if set1 is a subset of set2
result = set1.issubset(set2)
print(result)  # Output: True
```

4. **issuperset():** The issuperset() method in Python is used to check whether a set is a superset of another set. A set A is considered a superset of another set B if all elements of B are also present in A. As a shortcut, we can use the >= operator

```
set1 = {1, 2, 3, 4, 5}
set2 = {1, 2, 3}
# Check if set1 is a superset of set2
result = set1.issuperset(set2)
print(result)  # Output: True
```

5. **symmetric_difference_update():** Remove the items that are present in both sets, AND insert the items that is not present in both sets. As a shortcut, we can use the ^= operator

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.symmetric_difference_update(y)
print(x) # Output: {'cherry', 'google', 'microsoft', 'banana'}
```

6. update(): Insert the items from set 1 into set 2. As a shortcut, we can use the |= operator.

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
z = {"cherry", "micra", "bluebird"}
x.update(y, z)
print(x) # Output: {'apple', 'microsoft', 'cherry', 'banana',
'google', 'micra', 'bluebird'}
```

Join Method	Updated Method
The original method returns a new set containing the result of the operation without modifying the original set	The updated method modifies the original set directly to reflect the result of the operation.
It leaves the original set unchanged.	It updates the original set with the elements resulting from the operation.

For example, consider the difference() and difference_update() methods:

- difference() returns a new set containing the elements that are present in the original set but not in the specified set, without modifying the original set.
- difference_update() removes the elements of the specified set from the original set, effectively updating the original set with the difference.

Similarly, this distinction applies to other set methods such as intersection(), union(), symmetric_difference(), and their respective update methods.

Understand comparison between lists, tuples and sets

Aspect	Lists	Tuples	Sets
Mutability	Changeable	unchangeable	Changeable
Indexing	Yes	Yes	No
Duplicate	Allowed	Allowed	Not Allowed
Order	Ordered	Ordered	Unordered
Syntax	Enclosed in square	Enclosed in	Enclosed in curly

	brackets [ ]	parentheses ( )	braces { }

Understand what dictionaries are

A dictionary in Python is an unordered collection of items. Each item in a dictionary is stored as a key-value pair, where each key is unique and associated with a corresponding value. Dictionaries are mutable, meaning their elements can be modified after creation. Dictionaries do not allow duplicated keys, ensuring that each key remains unique within the dictionary.

Creating Dictionaries: Dictionaries are defined using curly braces {}, with key-value pairs separated by colons :. Each key-value pair is separated by commas.

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
```

In the dictionary my_dict, "name", "age", and "city" are keys, while "John", 30, and "New York" are their corresponding values.

```
my_dict = {1: 'apple', 2: 'banana', 3: 'cherry'}
```

In this dictionary my_dict, the keys are integers (1, 2, 3), and each key is associated with a corresponding value ('apple', 'banana', 'cherry').

Note: When you write s = {}, you're creating an empty dictionary, not an empty set. Therefore, when you try to use the add() method, which is specific to sets, on this dictionary, Python raises an AttributeError because dictionaries don't have an add() method.

In a dictionary, each key must be unique but the values can be duplicated. We can assign multiple values to a single key by using a list or another data structure as the value associated with that key. This allows you to store multiple values under a single key.

Note: When you define a dictionary with duplicate keys, only the last occurrence of the key-value pair is stored.

```
# Creating a dictionary with duplicate keys
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
```

```
  "year": 2020
}
print(thisdict)
```

The output will be:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2020}
```

```
# Creating a dictionary with duplicate values
scores = {
    "Alice": 85,
    "Bob": 75,
    "Charlie": 85,
    "David": 90,
    "Eve": 75
}
print(scores)
```

The output will be:

```
{'Alice': 85, 'Bob': 75, 'Charlie': 85, 'David': 90, 'Eve': 75}
```

```
thisdict = {
  "brand": "Ford",
  "electric": False,
  "year": 1964,
  "colors": ["red", "white", "blue"]
}
```

Tuples can indeed be used as keys in dictionaries in Python. Unlike lists, tuples are immutable, which means they cannot be changed after creation. This immutability makes tuples suitable for use as keys in dictionaries

```
# Create a dictionary with tuple keys
student_grades = {('John', 'Doe'): 85, ('Alice', 'Smith'): 92,
('Bob', 'Johnson'): 78}
```

The dict() Constructor: It is also possible to use the dict() constructor to make a dictionary.

```
thisdict = dict(name = "John", age = 36, country = "Norway")
print(thisdict)
```

The output will be:

```
{'name': 'John', 'age': 36, 'country': 'Norway'}
```

```
list_of_lists = [['a', 1], ['b', 2], ['c', 3]]
# Convert to dictionary
dict_from_lists = dict(list_of_lists)
print(dict_from_lists)
```

The output will be:

```
{'a': 1, 'b': 2, 'c': 3}
```

We can also use the type() function with dictionaries:

```
thisdict = {'name': 'John', 'age': 36, 'country': 'Norway'}
print(thisdict)
```

The output will be:

```
<class 'dict'>
```

Nested Dictionaries: A dictionary can contain dictionaries, this is called nested dictionaries.

```
# Nested dictionary example
student_records = {
    "John": {"age": 25, "grade": "A", "city": "New York"},
    "Alice": {"age": 23, "grade": "B", "city": "Los Angeles"},
    "Bob": {"age": 27, "grade": "B+", "city": "Chicago"}
}
```

n this example, student_records is a nested dictionary where each student's name ("John", "Alice", and "Bob") serves as a key, and their corresponding details (age, grade, and city) are stored as inner dictionaries.

Understand common methods and operations associated with dictinoaries

Length of a Dictinoary: To determine how many items a dictionary has, use the len() function.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "year": 2020
}
print(len(thisdict))
```

The output will be:

```
3
```

In this example, the key "year" appears twice in the dictionary, with different values ("1964" and "2020"). Since the last occurrence of the key "year" is associated with the value "2020", the dictionary effectively contains only three key-value pairs: "brand" -> "Ford", "model" -> "Mustang", and "year" -> 2020.

Accessing Elements: You can access the value associated with a key by using square brackets [ ] and specifying the key. If the key doesn't exist, it will raise a KeyError exception.

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
print(my_dict["name"])
```

The output will be:

```
John
```

Note: In Python, accessing a key that doesn't exist in a dictionary using square bracket notation ([]) will result in a KeyError.

```
# Create a dictionary with tuple keys
student_grades = {('John', 'Doe'): 85, ('Alice', 'Smith'): 92,
('Bob', 'Johnson'): 78}
# Accessing values using tuple keys
print(student_grades[('John', 'Doe')])
```

```
print(student_grades[('Alice', 'Smith')])
```

The output will be:

```
85
92
```

```
students = {
    "student1": {"name": "Alice", "age": 20, "major": "Computer
Science"},
    "student2": {"name": "Bob", "age": 22, "major":
"Engineering"},
    "student3": {"name": "Charlie", "age": 21, "major":
"Mathematics"}
}

# Accessing information about student2
print("Name:", students["student2"]["name"])
print("Age:", students["student2"]["age"])
print("Major:", students["student2"]["major"])
```

The output will be:

```
Name: Bob
Age: 22
Major: Engineering
```

There is also a method called get() that will give you the same result.

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
print(my_dict.get("name")) # Output: John
```

```
# Nested dictionary representing information about students
students = {
    "student1": {"name": "Alice", "age": 20, "major": "Computer
Science"},
    "student2": {"name": "Bob", "age": 22, "major":
```

```
"Engineering"},
    "student3": {"name": "Charlie", "age": 21}
}

# Accessing information about student2 using get() method
print("Name:", students.get("student2").get("name"))
print("Age:", students.get("student2").get("age"))
print("Major:", students.get("student3").get("major"))
```

The output will be:

```
Name: Bob
Age: 22
Major: None
```

```
d = {1:2, "abc":5, "def":7}
print(d.get(0,5)) # Output : 5
```

Note: If the key for which you're trying to retrieve a value is not present in the dictionary and the key is not present in the dictionary, get() will return None or return a custom desired value.

Dictionaries have several built-in methods such as keys(), values(), and items() to access keys, values, and key-value pairs respectively.

```
my_dict = {"name": "John", "age": 30, "city": "New York"}
print(my_dict.keys())
print(my_dict.values())
print(my_dict.items())
```

The output will be:

```
dict_keys(['name', 'age', 'city'])
dict_values(['John', 30, 'New York'])
dict_items([('name', 'John'), ('age', 30), ('city', 'New York')])
```

Note: The items() method returns a view object that displays a list of tuples. Each tuple contains a key-value pair from the dictionary.

setdefault(): setdefault() is a method of accessing items in a dictionary. It allows you to

retrieve the value associated with a specified key from the dictionary. If the key exists in the dictionary, setdefault() returns its corresponding value. If the key does not exist, setdefault() inserts the key with a default value into the dictionary and returns the default value. If no default value is specified, setdefault() will insert the key into the dictionary with a default value of None and return None.

Syntax:

```
dict.setdefault(key, default_value)
```

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = car.setdefault("model")
print(x)
```

The output will be:

```
Mustang
```

```
# Define a dictionary
my_dict = {'name': 'John', 'age': 30}
# Using setdefault() to add a new key-value pair
my_dict.setdefault('city', 'New York')
print(my_dict)
```

The output will be:

```
{'name': 'John', 'age': 30, 'city': 'New York'}
```

```
# Define a dictionary
my_dict = {'name': 'John', 'age': 30}
# Using setdefault() without specifying a default value
my_dict.setdefault('city')
print(my_dict)
```

The output will be:

```
{'name': 'John', 'age': 30, 'city': None}
```

Finding minimum or maximum element in a dictionary: To find the minimum or maximum element in a dictionary, you can use the min() and max() functions with appropriate parameters.

```
# Dictionary representing ages of people
ages = {'Alice': 25, 'Bob': 30, 'Charlie': 20}
# Finding the minimum age
min_age = min(ages.values())
print("Minimum age:", min_age)
max_age = max(ages.values())
print("Maximum age:", max_age)
```

The output will be:

```
Minimum age: 20
Maximum age: 30
```

Note: If the dictionary contains non-numeric values, such as strings or other objects, ensure that they support comparison operations. Otherwise, you may encounter a TypeError. If the dictionary is empty, calling min() or max() will result in a ValueError. Ensure that the dictionary is not empty before using these functions.

```
# Dictionary with string values
ages = {"Alice": 30, "Bob": 25, "Charlie": 35}
# Finding the minimum age
min_age = min(ages.values())
print("Minimum age:", min_age)
# Finding the maximum age
max_age = max(ages.values())
print("Maximum age:", max_age)
# Another dictionary with non-numeric values
grades = {"Alice": "A", "Bob": "B", "Charlie": "C"}
# Finding the minimum grade
min_grade = min(grades.values())
print("Minimum grade:", min_grade)
# Finding the maximum grade
max_grade = max(grades.values())
print("Maximum grade:", max_grade)
```

The output will be:

```
Minimum age: 25
Maximum age: 35
Minimum grade: A
Maximum grade: C
```

```
# Dictionary with mixed data types
data = {"Alice": 30, "Bob": "25", "Charlie": 35}
min_age = min(data.values())
max_age = max(data.values())
print("Minimum age:", min_age)
print("Maximum age:", max_age)
```

The output will be:

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

sum of all elements in a dictionary: To find the sum of all elements (values) in a dictionary, you can use the sum().

```
# Dictionary with numeric values
data = {"A": 10, "B": 20, "C": 30}
# Calculate the sum of all values in the dictionary
total_sum = sum(data.values())
# Print the total sum
print("Sum of all elements:", total_sum)
```

The output will be:

```
Sum of all elements: 60
```

Note: When using the sum() function with dictionary values, it's important to ensure that all the values are numeric (integers or floats). If the dictionary contains non-numeric values, such as strings or other non-numeric types, the sum() function will raise a TypeError.

```
# Dictionary with mixed data types
data = {"A": 10, "B": "20", "C": 30}
# Calculate the sum of all values in the dictionary
```

```
total_sum = sum(data.values())
# Print the total sum
print("Sum of all elements:", total_sum)
```

The output will be:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Change / add Dictionary Items: Keys in a dictionary are immutable, meaning they cannot be changed after they are created. Once a key-value pair is added to a dictionary, the key cannot be modified. However, you can add new key-value pairs or remove existing ones.When changing/ adding items in a dictionary in Python, there are two primary aspects:

1. **Direct Assignment:** This involves directly accessing an item in the dictionary by its key and assigning a new value to it. If the key already exists in the dictionary, its associated value is updated with the new value. If the key does not exist, a new key-value pair is added to the dictionary.

```
# Initial dictionary
my_dict = {"name": "John", "age": 30, "city": "New York"}
# Direct assignment
my_dict["age"] = 35  # Update existing item
my_dict["gender"] = "Male"  # Add new item
# Printing the updated dictionary
print(my_dict)
```

The output will be:

```
{'name': 'John', 'age': 35, 'city': 'New York', 'gender': 'Male'}
```

In this example, We directly access the item "age" in the dictionary my_dict and assign a new value 35 to it, updating the existing value. We also directly assign a new key-value pair "gender": "Male" to my_dict, adding a new item to the dictionary.

2. **Using the update() Method:** The update() method will update the dictionary with the items from the given argument. If the item does not exist, the item will be added. The argument must be a dictionary, or an iterable object with key:value pairs.

```
# Initial dictionary
person = {
```

```
    "name": "John",
    "age": 30,
    "city": "New York"
}
# Update the dictionary with new information
person.update({"age": 35, "city": "Los Angeles", "gender":
"Male"})
print(person)
```

The output will be:

```
{'name': 'John', 'age': 35, 'city': 'Los Angeles', 'gender':
'Male'}
```

```
# Initial dictionary
my_dict = {"name": "John", "age": 30, "city": "New York"}
# Dictionary to update with
update_dict = {"age": 35, "gender": "Male", "occupation":
"Engineer"}
# Using the update() method
my_dict.update(update_dict)
# Printing the updated dictionary
print(my_dict)
```

The output will be:

```
{'name': 'John', 'age': 35, 'city': 'New York', 'gender': 'Male',
'occupation': 'Engineer'}
```

zip(): The zip() function in Python is used to combine elements from two or more iterables (such as lists, tuples, or strings) into tuples.

```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
zipped = zip(list1, list2)
print(zipped)
```

When you print the zip() object directly, it will display the type of object and its memory location, indicating that it's an iterator.

The output will be:

```
<zip object at 0x1478bfcfa340>
```

To see the contents of the zip() object, you can convert it into a list/tuple/set/dictionary

```
zipped_list = list(zipped)
print(zipped_list)
```

The output will be:

```
[(1, 'a'), (2, 'b'), (3, 'c')]
```

```
keys = {1, 2, 3, 4, 5}
values = {'a', 'b', 'c', 'd', 'e'}
my_dict = dict(zip(keys, values))
print(my_dict)
```

The output will be:

```
{1: 'c', 2: 'd', 3: 'b', 4: 'a', 5: 'e'}
```

Remove Dictionary Items: There are various methods available in Python to remove items from a dictionary

1. **Using the del keyword:** Deletes the item with the specified key.

```
# Initial dictionary
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
# Removing the "age" key and its corresponding value
del person["age"]
print(person)
```

The output will be:

```
{'name': 'John', 'city': 'New York'}
```

The del keyword can also delete the dictionary completely

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del thisdict
print(thisdict)
```

This will cause an error because "thisdict" no longer exists.

The output will be:

```
NameError: name 'thisdict' is not defined
```

2. **Using the pop() method:** Removes the item with the specified key and returns its value.

```
# Initial dictionary
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}

# Removing the value associated with the "age" key
removed_age = person.pop("age")

print(person)
print("Removed age:", removed_age)
```

The output will be:

```
{'name': 'John', 'city': 'New York'}
Removed age: 30
```

If we don't specify any key in the pop() method, it will raise a TypeError because the pop() method requires at least one argument, which is the key of the item to be removed. Without specifying a key, the method doesn't know which item to remove from the dictionary.

3. **popitem():** Removes and returns the last inserted item (key-value pair) as a tuple.

```
# Initial dictionary
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
# Removing the last inserted item
removed_item = person.popitem()
print(person)
print("Removed item:", removed_item)
```

The output will be:

```
{'name': 'John', 'age': 30}
Removed item: ('city', 'New York')
```

4. **clear():** Removes all items from the dictionary.

```
# Initial dictionary
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
# Clearing the dictionary
person.clear()
print(person)
```

The output will be:

```
{}
```

Copy Dictionaries: There are some methods you can use to copy items from one dictionary to another in Python.

1. **Using the copy() method:** This method creates a shallow copy of the dictionary.

```
# Original dictionary
original_dict = {"name": "Alice", "age": 25, "city": "New York"}
# Copying the items to a new dictionary
copied_dict = original_dict.copy()
# Modifying the copied dictionary
copied_dict["name"] = "Bob"
# Printing both dictionaries
print("Original dictionary:", original_dict)
print("Copied dictionary:", copied_dict)
```

The output will be:

```
Original dictionary: {'name': 'Alice', 'age': 25, 'city': 'New
York'}
Copied dictionary: {'name': 'Bob', 'age': 25, 'city': 'New York'}
```

2. **Using assignment:** Simply assigning one dictionary to another variable creates a reference to the original dictionary.

```
# Original dictionary
original_dict = {"name": "Alice", "age": 25, "city": "New York"}
# Assigning the original dictionary to a new variable
copied_dict = original_dict
# Modifying the copied dictionary
copied_dict["name"] = "Bob"
# Printing both dictionaries
print("Original dictionary:", original_dict)
print("Copied dictionary:", copied_dict)
```

The output will be:

```
Original dictionary: {'name': 'Bob', 'age': 25, 'city': 'New
York'}
Copied dictionary: {'name': 'Bob', 'age': 25, 'city': 'New York'}
```

3. **Using dict() constructor:** We can also use the dict() constructor to create a copy of a dictionary.

```
# Original dictionary
original_dict = {"name": "Alice", "age": 25, "city": "New York"}
# Using the dict() constructor
copied_dict_3 = dict(original_dict)
# Modifying the copied dictionary
copied_dict_3["name"] = "Charlie"
# Printing all dictionaries
print("Original dictionary:", original_dict)
print("Copied dictionary (dict() constructor):", copied_dict_3
```

The output will be:

```
Original dictionary: {'name': 'Alice', 'age': 25, 'city': 'New
York'}
Copied dictionary (dict() constructor): {'name': 'Charlie', 'age':
25, 'city': 'New York'}
```

fromkeys(): The fromkeys() method in Python is used to create a new dictionary with keys from a specified iterable and values set to a default value (which defaults to None if not provided).

Syntax:

```
dict.fromkeys(iterable[, value])
```

```
# Create a dictionary with keys from a list and default value set
to 0
keys = ["a", "b", "c"]
default_value = 0
new_dict = dict.fromkeys(keys, default_value)
print(new_dict)
```

The output will be:

```
{'a': 0, 'b': 0, 'c': 0}
```

```
my_string = "hello"
my_set = set(my_string)
my_dict = dict.fromkeys(my_set, )
print(my_dict)
```

The output will be:

```
{'l': None, 'h': None, 'o': None, 'e': None}
```

Understand comparison between lists, tuples, sets and dictionaries

Feature	Lists	Tuples	Sets	Dictionaries
Definition	Ordered collection of items	Ordered collection of items	Unordered collection of unique items	Unordered collection of key-value pairs
Mutability	Changeable	Unchangeable	Changeable	Changeable
Syntax	Square brackets []	Parentheses ()	Curly braces {}	Curly braces {}
Indexing	Supports indexing and slicing	Supports indexing and slicing	Does not support indexing	Supports key-based access
Duplication	Allows duplicate elements	Allows duplicate elements	Does not allow duplicate elements	Keys must be unique
Initialization	my_list = [1, 2, 3]	my_tuple = (1, 2, 3)	my_set = {1, 2, 3}	my_dict = {'a': 1, 'b': 2, 'c': 3}
Performance	Slower than sets/dicts for lookups & okay for sequential access.	Faster than lists for iteration (immutable), but still slower than sets/dicts for lookups.	Fastest for membership tests due to hashing but slow in iteration.	Fast key-based access, especially for large datasets but slow in iteration.
Use Cases	When the order of elements matters	When data should not be changed after creation	When uniqueness is required	When data needs to be stored with keys

codingninjas